

Programação Estruturada - especificação do trabalho

Especificação

O trabalho é composto por até três estudantes e o grupo deve implementar um capítulo de um livro-jogo.

-

Para isso, será necessário escolher um capítulo de um dos livros, ou então uma certa quantidade de páginas que torne o programa jogável, e implementar o livro-jogo no modo console. O texto do capítulo deve estar dentro de um arquivo e o programa deve carregar o capítulo em memória. A medida que o leitor decide que decisão tomar, o programa deve seguir o respectivo fluxo e carregar o novo diálogo.

Alternativamente, o grupo pode escolher escrever a sua própria história. Nesse caso, basta apresentar o início da história com uma quantidade razoável de possíveis caminhos que seriam equivalentes ao tamanho de um capítulo de um livro.

Avaliação

Conteúdo:

- Código fonte em linguagem C da simulação do livro-jogo.
- O jogo-livro deve ser desenvolvido no modo texto.

CrITÉRIOS de avaliação no código:

- Clareza do código, inclusive com indentação correta
- Laços (`for`, `while` e `breaks`)
- Estrutura procedural do código
- Declaração e escopo das variáveis
- Conhecimento do discente sobre o código que o grupo está apresentado ¹

¹ Vender gato por lebre ou encher linguiça perde ponto

Requisitos:

Um programa que simula um livro-jogo típico deveria ter:

1. **Tela inicial**, onde o usuário pode escolher entre:
 - Iniciar um novo jogo.
 - Continuar um jogo gravado.
 - Ranking de pontuações.
 - Listar integrantes do grupo.
2. **Tela de criação**, onde o usuário pode criar seu personagem.
 - Geração aleatório de atributos.
 - Escolha dos atributos com distribuição de pontos.
3. **Tela do jogo**, contendo:
 - Um menu descrevendo as opções que o jogador pode realizar, por exemplo, usar uma poção de cura, gravar o jogo, etc.
 - Descrição da situação atual o personagem e quais opções ele tem disponível para realizar naquele momento.
4. **Tela de combate**, contendo:
 - Informação do jogador e do inimigo.
 - Opções das ações que o jogador pode realizar em cada rodada da batalha (lutar, fugir, conversar, etc).
5. **Tela final**, caso o personagem chegue a um final no livro-jogo, podendo:
 - Encerrar o jogo e possível gravação da pontuação na tabela de recordes.
 - Recomeçar do último ponto de gravação.

O programa deve utilizar também uma estrutura **Personagem** contendo informações dos personagens (tanto jogador quanto de inimigos). Essa estrutura pode conter, por exemplo, informações como:

- **Nome**
- **Local** (onde o personagem no andamento da aventura no livro-jogo)
- **Pontuação**
- **Alinhamento** (bondoso, neutro ou maldoso) que pode variar dependendo das ações tomadas pelo personagem no jogo e que pode afetar as opções do jogo no futuro.
- **Informações básicas:**
 - Força
 - Inteligência

Sabedoria

Carisma

etc...

- **Informações descritivas:**

Vida

Magia (dependendo do jogo)

Energia

- **Itens:** como um vetor de inteiros, onde cada posição corresponde a quantidade de itens que o usuário obteve).
- **Quests:** como um vetor de inteiros que armazena quais quests o jogador fez durante o jogo. Essas quests podem influenciar em decisões tomadas no meio do jogo, por exemplo, se o jogador foi na biblioteca e estudou sobre uma fábrica, ficará marcada essa quest como realizada e, no futuro, quando visitar essa fábrica, isso pode ajudar o jogador fornecendo uma nova escolha mais fácil que esse obteve usando esse conhecimento.

Seria interessante o programa conter funções como:

- **void grava(Personagem)**, que recebe as informações atuais do personagem e grava o estado atual do jogo em um arquivo.
- **void le(Personagem)**, que restaura o jogo ao estado dado pelas informações gravadas no arquivo.
- **void recorde(Personagem)**, que grava, caso seja necessário, a pontuação do personagem na lista de recordes.
- **void inicio()**, que mostra a tela de início do jogo.
- **void criar(Personagem*)**, que recebe o jogador como parâmetro, abre a tela de criação do jogador, cria o personagem e retorna a tela do jogo (no início do jogo).
- **void menu()**, que imprime na tela as opções que o jogador pode tomar naquele determinado local e estado do jogo.
- **void combate (Personagem*, Personagem*)**, que recebe o jogador e o inimigo como parâmetros, abre a tela do combate, realizar o combate e, ao fim, retorna a tela do jogo.
- **void final()**, que vai para a tela final do jogo, com opções de encerrar o jogo e gravar o recorde ou continuar o jogo no último local gravado.